Quiz # 3 (Data Structures) CLO3 **BS DS Fall 2025 Morning**

Name:_____ **Roll number:_____**

Time Allowed: 20 min **Total Marks: 30**

Q1. Order the Big O terms in ascending order.
n^3, n, 1 , log n, n/2, $n^{1.01}$, n log n, 2^n, 1.01^n,n^2 log n

1	_____1_____	6	____ $n^{1.01}$____
2	____log n_____	7	___ n^2 log n __
3	_____n/2_____	8	_____ n^3_____
4	____n_____	9	___1.01^n _____
5	____n log n_____	10	____2^n _____

Q2. A pre-fix expression (in a string/char format) has single digit integers only. Using stack evaluate the pre-fix expression. Write down the following function:

int prefix_evaluator(string exp) {

```
stack<int> st;
 for (int i = exp.length() - 1; i >= 0; i--) {
        char c=exp[i];
        if (c >= '0' && c <= '9')
                st.push(c – '0');                    //'0' is subtracted to make character to digit
        else
        {
                n2 = st.front(), st.pop();

                n1 = st.front(), st.pop();

                if (c == '+')
                        st.push(n1 + n2);
                else if (c == '-')
                        st.push(n1 - n2);
                else if (c == '*')
                        st.push(n1 * n2);
                else if (c == '/')
                        st.push(n1 / n2);
                else
                         st.push(n1 % n2);
        }
}
cout << st.top() ; //stack top has result at the end
```

Q3. An integer array has numbers 0 to 9 only. A duplicate removal consists of choosing two adjacent and equal numbers and removing them. We repeatedly make duplicate removals until we no longer can. Return the final array of values after all such duplicate removals have made. Sample input/ output given in the box.

void duplicateRemove(int *arr, int n, int *result, int &resultSize);

```
Input: {3,5,5,3,8,3}
Output: {8,3}
Input: {3,7,5,5,7,4}
Output: {3,4}
```

```
void duplicateRemove(int *arr, int n, int *result, int &resultSize)
{
   Stack<int> s(n);

   for (int i = 0; i < n; i++)
   {
      if (!s.isEmpty() && s.stackTop() == arr[i])
      {
         s.pop();
      }
      else
      {
         s.push(arr[i]);
      }
   }

   resultSize = 0;

   // Copy stack elements into result
   while (!s.isEmpty())
   {
      result[resultSize] = s.pop();
      resultSize++;
   }

   // Reverse result because stack is LIFO
   for (int i = 0; i < resultSize / 2; i++)
   {
      int temp = result[i];
      result[i] = result[resultSize - 1 - i];
      result[resultSize - 1 - i] = temp;
   }
}
```